

torch.fft.fftfreq#

<https://pytorch.org/docs/stable/generated/torch.fft.fftfreq.html>

Description:

Computes the discrete Fourier Transform sample frequencies for a signal of size n . By convention, `fft()` returns positive frequency terms first, followed by the negative frequencies in reverse order, so that $f[-i]$ for all 0

Function Signature

```
torch.fft.fftfreq(n, d=1.0, *, out=None, dtype=None,
                  layout=torch.strided, device=None,
                  requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). layout (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Code Examples

```
f = [0, 1, ..., (n - 1) // 2, -(n // 2), ..., -1] / (d * n)
```

```
>>> torch.fft.fftfreq(5)
tensor([ 0.0000,  0.2000,  0.4000, -0.4000, -0.2000])
```

```
>>> torch.fft.fftfreq(4)
tensor([ 0.0000,  0.2500, -0.5000, -0.2500])
```

Notes & Warnings

NOTE

By convention, `fft()` returns positive frequency terms first, followed by the negative frequencies in reverse order, so that `f[-i]` for all $0 < i \leq n/2$ in Python gives the negative frequency terms. For an FFT of length n and with inputs spaced in length unit d , the frequencies are: $f = [0, 1, \dots, (n - 1) // 2, -(n // 2), \dots, -1] / (d * n)$

NOTE

For even lengths, the Nyquist frequency at `f[n/2]` can be thought of as either negative or positive. `fftfreq()` follows NumPy's convention of taking it to be negative.

Detailed Documentation

Documentation

Computes the discrete Fourier Transform sample frequencies for a signal of size n .

By convention, `fft()` returns positive frequency terms first, followed by the negative frequencies in reverse order, so that `f[-i]` for all $0 < i \leq n/2$ in Python gives the negative frequency terms. For an FFT of length n and with inputs spaced in length unit d , the frequencies are:

For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. `fftfreq()` follows NumPy's convention of taking it to be negative.

`n` (int) – the FFT length

`d` (float, optional) – The sampling length scale. The spacing between individual samples of the FFT input. The default assumes unit spacing, dividing that result by the actual spacing gives the result in physical frequency units.

`out` (Tensor, optional) – the output tensor.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`).

`layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

For even input, we can see the Nyquist frequency at $f[2]$ is given as negative:

